Lecture 02: C++ Foundations for Data Structures

TKU211231 — Student Notes

Syukron Abu Ishaq Alfarozi
Department of Electrical and Information Engineering
Universitas Gadjah Mada

February 19, 2026

Contents

1 How C++ Programs Are Built **3**
1.1 From Source Code to Executable . . . 3
1.2 Header Files . . . 4

2 Memory: Stack vs Heap **6**
2.1 Stack Memory . . . 6
2.2 Heap Memory (Free Store) . . . 7

3 Pointers: Understanding Memory Addresses **8**
3.1 What is a Pointer? . . . 8
3.2 Pointer Syntax . . . 9
3.3 Pointers and Arrays . . . 10
3.4 Null and Dangling Pointers . . . 11

4 References: Aliases for Variables **12**
4.1 Parameter Passing: Value, Reference, Pointer . . . 13

5 Dynamic Memory: `new` and `delete` **16**
5.1 Allocating Memory with `new` . . . 16
5.2 Freeing Memory with `delete` . . . 17
5.3 Memory Leaks . . . 18

6 Classes and Objects **19**
6.1 From Struct to Class . . . 19
6.2 Creating a Proper Class . . . 20
6.3 Classes with Dynamic Memory . . . 22
6.4 The Rule of Three . . . 24

7 Templates: Generic Programming **26**
7.1 Function Templates . . . 27
7.2 Class Templates . . . 28

8 Building a Node Structure **30**
8.1 What is a Node? 30
8.2 Creating and Linking Nodes 31

9 Summary and Key Takeaways **34**
9.1 Common Pitfalls Checklist 35
9.2 Practice Problems - Solutions 35

10 What's Next? **36**

Learning Objectives

By the end of this lecture, you should be able to:

1. Understand how C++ programs are compiled from source code to executable
2. Explain the difference between stack and heap memory
3. Use pointers to access and modify memory addresses
4. Distinguish between pass-by-value, pass-by-reference, and pass-by-pointer
5. Allocate and deallocate dynamic memory using `new` and `delete`
6. Identify and prevent common memory errors (leaks, dangling pointers)
7. Create classes with proper constructors and destructors
8. Understand the Rule of Three for classes managing resources
9. Write generic code using templates
10. Build a basic `Node` structure for linked data structures

Story

Before a carpenter builds a house, she masters her tools—saw, hammer, level. She doesn't try to learn carpentry and master tools at the same time. She starts with the tools.
C++ is our tool for building data structures. You've learned basic C++ before, but today we go deeper. We'll learn about *pointers*, *memory management*, and *classes*—the essential tools you need to build efficient data structures.
Don't worry if these concepts feel challenging at first. Pointers are often considered one of the hardest topics in programming, but once you understand them, you'll have a superpower that many programmers lack. Let's begin!

1 How C++ Programs Are Built

1.1 From Source Code to Executable

When you write a C++ program, you create **source files** (files ending in `.cpp`). But computers can't directly run these files. They need to be *compiled* into machine code.

Key Concept

The C++ build process has three main steps:

1. **Preprocessing**: The preprocessor handles directives like `#include` and `#define`
2. **Compiling**: Each `.cpp` file is compiled into an *object file* (`.o` or `.obj`)
3. **Linking**: All object files are combined into one executable program

`main.cpp` → `main.o` → `program.exe`

Example

To compile a simple C++ program:

```
# Compile and link in one step
g++ -std=c++17 -Wall -o myprogram main.cpp

# Or compile and link separately
g++ -std=c++17 -c main.cpp       # Creates main.o
g++ -o myprogram main.o          # Creates executable
```

Common flags:

- `-std=c++17`: Use C++17 standard
- `-Wall`: Show all warnings (helps catch bugs)
- `-o filename`: Name the output file
- `-c`: Compile only, don't link

1.2 Header Files

For larger projects with multiple files, we use **header files** (`.h` or `.hpp`) to declare functions and classes, and **source files** (`.cpp`) to define them.

Example

math_utils.h (header file):

```
#ifndef MATH_UTILS_H
#define MATH_UTILS_H

int add(int a, int b);  // Declaration
int multiply(int a, int b);

#endif
```

math_utils.cpp (source file):

```
#include "math_utils.h"

int add(int a, int b) {        // Definition
    return a + b;
}

int multiply(int a, int b) {
    return a * b;
}
```

main.cpp:

```
#include <iostream>
#include "math_utils.h"  // Use quotes for your own headers

int main() {
    std::cout << add(5, 3) << "\n";
    return 0;
}
```

To compile and run this multi-file project:

```
g++ -std=c++17 -Wall -o myprogram main.cpp math_utils.cpp
./myprogram
```

Or compile separately then link:

```
g++ -std=c++17 -Wall -c math_utils.cpp  # Creates math_utils.o
g++ -std=c++17 -Wall -c main.cpp        # Creates main.o
g++ -o myprogram math_utils.o main.o    # Link together
./myprogram
```

Key Concept

Always use **include guards** (`#ifndef`, `#define`, `#endif`) in header files to prevent the same file from being included multiple times. Alternatively, use `#pragma once` at the top of the header.

2 Memory: Stack vs Heap

Story

Imagine your computer's memory as a huge apartment building with millions of rooms (each room is one byte). When your program runs, it gets some space in this building. But not all spaces work the same way!

2.1 Stack Memory

Key Concept

Stack memory is like a stack of plates:

- Automatically managed by the compiler
- Used for local variables in functions
- Memory is automatically freed when a function returns
- Very fast
- Limited in size

```cpp
#include <iostream>
using namespace std;

void someFunction() {
    int x = 10;       // x is on the stack
    double y = 3.14; // y is on the stack
    cout << x + y << "\n";
}  // x and y are automatically destroyed here

int main() {
    someFunction();
    // x and y no longer exist here
    return 0;
}
```

Listing 1: Stack memory example

To compile and run:

```
g++ -std=c++17 -Wall -o stack_demo stack_demo.cpp
./stack_demo
```

2.2 Heap Memory (Free Store)

Key Concept

Heap memory is like renting a storage unit:

- Manually managed by the programmer
- You decide when to allocate (`new`) and deallocate (`delete`)
- Memory persists until you explicitly free it
- Slower than stack, but much larger
- Used when you don't know size at compile time

```cpp
#include <iostream>
using namespace std;

int main() {
    int* p = new int(42);  // Allocate an int on the heap
    cout << *p << "\n";    // Output: 42

    delete p;              // Free the memory
    p = nullptr;           // Good practice

    return 0;
}
```

Listing 2: Heap memory example

To compile and run:

```
g++ -std=c++17 -Wall -o heap_demo heap_demo.cpp
./heap_demo
```

Key Concept

Visual Model of Memory:

```
High Address
+-------------------------+
|        Stack            |  <- Grows downward
|   (local variables)     |     Fast, automatic
|                         |
+-------------------------+
|                         |
|      (free space)       |
|                         |
+-------------------------+
|        Heap             |  <- Grows upward
|   (dynamic allocation)  |     Manual, flexible
|                         |
+-------------------------+
|   Global/Static Data    |
+-------------------------+
|   Code (Your program)   |
+-------------------------+
Low Address
```

3 Pointers: Understanding Memory Addresses

Story

Think of a pointer like a GPS coordinate. The coordinate itself isn't a house—it just tells you where the house is located. You can write the coordinate on paper (copy the pointer), follow it to visit the house (dereference), or lose the paper entirely (lose the pointer).
Pointers are one of the most powerful—and initially confusing—features of C++. Let's break them down step by step.

3.1 What is a Pointer?

Definition 3.1 (Pointer). *A **pointer** is a variable that stores the memory address of another variable.*

***Key operators**:*

- *`&variable`: Get the address of a variable ("address-of")*
- *`*pointer`: Access the value at an address ("dereference")*

```cpp
#include <iostream>
using namespace std;

```

```cpp
int main() {
    int x = 42;          // Regular variable
    int* p = &x;         // p is a pointer to int, stores address of x

    cout << "Value of x: " << x << "\n";            // 42
    cout << "Address of x: " << &x << "\n";         // 0x7ffd... (some address)
    cout << "Value of p: " << p << "\n";            // Same address as &x
    cout << "Value pointed to by p: " << *p << "\n";  // 42

    *p = 100;           // Change the value at address p (modifies x!)
    cout << "New value of x: " << x << "\n";        // 100

    return 0;
}
```

Listing 3: Basic pointer operations

To compile and run:

```
g++ -std=c++17 -Wall -o pointers pointers.cpp
./pointers
```

Key Concept

Visual Model:

```
Memory:
Address     Variable    Value
0x1000      x           42
0x2000      p           0x1000  (stores address of x)

Operations:
  x     -> 42           (normal access)
  &x    -> 0x1000       (address of x)
  p     -> 0x1000       (value stored in p)
  *p    -> 42           (follow address to get value)

After *p = 100:
  x = 100
  *p = 100  (same location)
```

3.2 Pointer Syntax

```cpp
int* p1;            // Pointer to int (uninitialized - DANGEROUS!)
int* p2 = nullptr; // Pointer to int, initialized to null (SAFE)

int x = 10;
```

```
int* p3 = &x;       // p3 points to x

double d = 3.14;
double* p4 = &d;    // p4 points to d

// Common mistake: mismatched types
// int* p5 = &d;    // ERROR! Can't assign double* to int*
```

Listing 4: Declaring and using pointers

Warning

Always initialize pointers!

```
int* p;          // Uninitialized - points to random memory!
*p = 42;         // CRASH! Undefined behavior

int* q = nullptr;  // Good: explicitly no address
if (q != nullptr) {
    *q = 42;        // Safe: won't execute
}
```

Rule: If you don't have a valid address to assign, use `nullptr`.

3.3 Pointers and Arrays

Key Concept

In C++, array names can be treated as pointers to the first element.

```
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {10, 20, 30, 40, 50};

    // arr is like a pointer to arr[0]
    int* p = arr;      // Same as: int* p = &arr[0];

    cout << arr[0] << "\n";    // 10
    cout << *p << "\n";        // 10 (same thing)

    cout << arr[1] << "\n";    // 20
    cout << *(p + 1) << "\n"; // 20 (pointer arithmetic)

    // Looping with pointers
    for (int i = 0; i < 5; i++) {
        cout << *(p + i) << " ";  // Equivalent to arr[i]
    }
    cout << "\n";

```

```
    return 0;
}
```

Listing 5: Pointers and arrays

To compile and run:

```
g++ -std=c++17 -Wall -o ptr_array ptr_array.cpp
./ptr_array
```

Key Concept

Pointer Arithmetic:
When you add 1 to a pointer, it moves forward by the size of the type:

- `int* p; p + 1` advances by 4 bytes (size of int)
- `double* q; q + 1` advances by 8 bytes (size of double)

```
int arr[5] = {10, 20, 30, 40, 50};

Address:  0x100  0x104  0x108  0x10C  0x110
Value:    [10]   [20]   [30]   [40]   [50]
          arr    arr+1  arr+2  arr+3  arr+4
          *arr   *(arr+1) ...
```

3.4 Null and Dangling Pointers

Warning

Two Deadly Pointer Errors:
1. Null Pointer Dereference:

```
int* p = nullptr;
*p = 10;            // CRASH! Segmentation fault
```

Always check before dereferencing:

```
int* p = nullptr;
if (p != nullptr) {
    *p = 10;
}
```

2. Dangling Pointer (pointer to deleted memory):

```
int* p = new int(5);
delete p;
*p = 10;            // CRASH! Undefined behavior
p = nullptr;        // Should do this after delete!
```

Golden Rule: After `delete`, immediately set pointer to `nullptr`.

Practice Exercise

Exercise 1: What does this code print?

```
int x = 5;
int* p = &x;
*p = *p + 10;
cout << x;
```

Exercise 2: Find the bug:

```
int* p;
*p = 100;
cout << *p;
```

Exercise 3: What's wrong here?

```
int* p = new int(42);
p = new int(99);
delete p;
```

4 References: Aliases for Variables

Key Concept

A **reference** is an *alias* (another name) for an existing variable. Once created, a reference cannot be changed to refer to a different variable.
Declaration: `int& ref = x;`

```
#include <iostream>
using namespace std;

int main() {
    int x = 10;

    // Reference - becomes another name for x
    int& ref = x;
    ref = 20;
    cout << "x = " << x << "\n";    // 20 (x was modified)

    // Pointer - stores address of x
    int* ptr = &x;
    *ptr = 30;
    cout << "x = " << x << "\n";    // 30 (x was modified)

    return 0;
}
```

Listing 6: References vs Pointers

To compile and run:

```
g++ -std=c++17 -Wall -o ref_vs_ptr ref_vs_ptr.cpp
./ref_vs_ptr
```

Key Concept

Differences Between Pointers and References:

Feature	Pointer	Reference
Declaration	`int* p = &x;`	`int& r = x;`
Access value	`*p`	`r`
Can be null?	Yes (`nullptr`)	No
Can be reassigned?	Yes	No
Must be initialized?	No	Yes

4.1 Parameter Passing: Value, Reference, Pointer

This is one of the most important concepts in C++!

```cpp
#include <iostream>
using namespace std;

// 1. Pass by VALUE - makes a copy
void byValue(int x) {
    x = 99;  // Only changes the local copy
}

// 2. Pass by REFERENCE - passes alias
void byReference(int& x) {
    x = 99;  // Changes the original!
}

// 3. Pass by POINTER - passes address
void byPointer(int* x) {
    *x = 99;  // Changes the original!
}

int main() {
    int num = 0;

    byValue(num);
    cout << "After byValue: " << num << "\n";       // 0 (unchanged)

    byReference(num);
    cout << "After byReference: " << num << "\n";  // 99 (changed!)

    num = 0;
    byPointer(&num);
    cout << "After byPointer: " << num << "\n";    // 99 (changed!)
```

```

    return 0;
}
```

Listing 7: Three ways to pass parameters

To compile and run:

```
g++ -std=c++17 -Wall -o pass_params pass_params.cpp
./pass_params
```

Output:

```
After byValue: 0
After byReference: 99
After byPointer: 99
```

Example

Classic Example: Swap Function

```cpp
// WRONG - doesn't work!
void swap(int a, int b) {
    int tmp = a;
    a = b;
    b = tmp;
}  // a and b are destroyed here - original unchanged!

// CORRECT - using references
void swap(int& a, int& b) {
    int tmp = a;
    a = b;
    b = tmp;
}

// CORRECT - using pointers
void swap(int* a, int* b) {
    int tmp = *a;
    *a = *b;
    *b = tmp;
}

int main() {
    int x = 1, y = 2;

    swap(x, y);       // Using reference version
    // Now x = 2, y = 1

    swap(&x, &y);     // Using pointer version
    // Now x = 1, y = 2
}
```

To compile and run:

```
g++ -std=c++17 -Wall -o swap swap.cpp
./swap
```

Key Concept

When to use which?

- **Pass by value**: Small types (int, char) when you don't need to modify
- **Pass by reference**: When you need to modify the original, or to avoid copying large objects
- **Pass by pointer**: When `nullptr` is a valid option, or when working with arrays

Practice Exercise

Exercise 4: Predict the output:

```
void mystery(int& x, int y) {
    x = x + y;
    y = x + y;
}

int main() {
    int a = 5, b = 10;
    mystery(a, b);
    cout << a << " " << b;
}
```

5 Dynamic Memory: `new` and `delete`

Story

Imagine you're organizing a party but don't know how many guests will come. You can't set up exactly 50 chairs in advance—you might need 10 or you might need 100!

In programming, we often don't know how much data we'll need until the program runs. That's what *dynamic memory allocation* solves. You rent memory when you need it and return it when you're done.

5.1 Allocating Memory with `new`

Key Concept

Single Object:

```
int* p = new int;          // Allocate uninitialized int
int* q = new int(42);      // Allocate int with value 42
double* r = new double(3.14);
```

Array:

```
int* arr = new int[10];          // Array of 10 ints
double* data = new double[100]; // Array of 100 doubles
```

5.2 Freeing Memory with `delete`

Warning

CRITICAL RULES:

1. Every `new` must have a matching `delete`
2. Use `delete` for single objects: `delete p;`
3. Use `delete[]` for arrays: `delete[] arr;`
4. NEVER mix them up!
5. Set pointer to `nullptr` after delete

```cpp
#include <iostream>
using namespace std;

int main() {
    // Single object
    int* p = new int(42);
    cout << *p << "\n";
    delete p;           // Free single object
    p = nullptr;        // Good practice

    // Array
    int size = 5;
    int* arr = new int[size];
    for (int i = 0; i < size; i++) {
        arr[i] = i * 10;
    }

    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << "\n";

    delete[] arr;       // Free array (note the [])
    arr = nullptr;

    return 0;
}
```

Listing 8: Proper new/delete usage

To compile and run:

```
g++ -std=c++17 -Wall -o dynamic_array dynamic_array.cpp
./dynamic_array
# Enter a number when prompted, e.g., 5
```

5.3 Memory Leaks

Warning

Memory Leak = allocated memory that you can never free because you lost the pointer!

```
// Leak 1: Forgot to delete
void leak1() {
    int* p = new int(100);
    // ... use p ...
    return;  // p is destroyed, but the int still exists!
}

// Leak 2: Lost the only pointer
void leak2() {
    int* p = new int(1);
    p = new int(2);       // Lost pointer to first int!
    delete p;             // Only frees second int
}

// Leak 3: Exception before delete
void leak3() {
    int* data = new int[1000];
    // If error happens here, delete never executes!
    // ... risky code ...
    delete[] data;
}
```

Listing 9: Common memory leak patterns

Effects of memory leaks:

- Program uses more and more memory over time
- Eventually runs out of memory and crashes
- Especially bad in long-running programs (servers, embedded systems)

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "How many numbers? ";
    cin >> n;

    // Allocate array of user-specified size
    int* numbers = new int[n];

    // Fill array
    for (int i = 0; i < n; i++) {
        numbers[i] = i * i;
```

```
    }

    // Print array
    for (int i = 0; i < n; i++) {
        cout << numbers[i] << " ";
    }
    cout << "\n";

    // Free memory
    delete[] numbers;
    numbers = nullptr;

    return 0;
}
```

Listing 10: Correct dynamic array example

Practice Exercise

Exercise 5: Fix the memory leaks:

```
void process() {
    int* data = new int[100];
    for (int i = 0; i < 100; i++) {
        data[i] = i;
    }
    cout << data[50];
    // What's missing?
}
```

Exercise 6: What's wrong with this code?

```
int* p = new int[10];
// ... use p ...
delete p;  // Bug!
```

6 Classes and Objects

Story

In the real world, a "student" isn't just a name or an ID number—it's a bundle of information: name, ID, GPA, courses, etc. Plus, there are things a student can *do*: enroll in a course, take an exam, etc.
C++ `class` lets us bundle data and functions together into one package. This is the foundation of *object-oriented programming*.

6.1 From Struct to Class

```
#include <iostream>
```

```cpp
#include <string>
using namespace std;

struct Student {
    string name;
    int id;
    double gpa;
};

int main() {
    Student s;
    s.name = "Ahmad";
    s.id = 12345;
    s.gpa = 3.75;

    cout << s.name << " has GPA " << s.gpa << "\n";

    return 0;
}
```

Listing 11: Starting with a struct

To compile and run:

```
g++ -std=c++17 -Wall -o student_struct student_struct.cpp
./student_struct
```

Key Concept

Struct vs Class in C++:

- `struct`: Members are `public` by default
- `class`: Members are `private` by default
- Otherwise, they're identical!

Convention:

- Use `struct` for simple data containers
- Use `class` for objects with behavior and encapsulation

6.2 Creating a Proper Class

```cpp
#include <iostream>
#include <string>
using namespace std;

class Student {
private:
    // Data members (private)
```

```cpp
    string name_;
    int id_;
    double gpa_;

public:
    // Constructor
    Student(string name, int id, double gpa)
        : name_(name), id_(id), gpa_(gpa) {
        cout << "Student created: " << name_ << "\n";
    }

    // Destructor
    ~Student() {
        cout << "Student destroyed: " << name_ << "\n";
    }

    // Getters (const means doesn't modify object)
    string getName() const { return name_; }
    int getId() const { return id_; }
    double getGpa() const { return gpa_; }

    // Setter with validation
    void setGpa(double gpa) {
        if (gpa >= 0.0 && gpa <= 4.0) {
            gpa_ = gpa;
        } else {
            cout << "Invalid GPA!\n";
        }
    }

    // Member function
    void print() const {
        cout << name_ << " (ID: " << id_
             << ") GPA: " << gpa_ << "\n";
    }
};

int main() {
    Student s1("Ahmad", 12345, 3.75);
    s1.print();

    s1.setGpa(3.90);
    s1.print();

    s1.setGpa(5.0);  // Invalid - rejected

    return 0;
}  // Destructor called automatically here
```

Listing 12: A C++ class with encapsulation

To compile and run:

```
g++ -std=c++17 -Wall -o student_class student_class.cpp
./student_class
```

Expected output:

```
Student created: Ahmad
Ahmad (ID: 12345) GPA: 3.75
Ahmad (ID: 12345) GPA: 3.9
Invalid GPA!
Student destroyed: Ahmad
```

Key Concept

Key Class Components:

- **Constructor**: Special function called when object is created
  - Same name as class
  - No return type
  - Can have parameters
- **Destructor**: Special function called when object is destroyed
  - Name is `~ClassName`
  - No parameters, no return type
  - Cleans up resources
- **Private members**: Only accessible within the class
- **Public members**: Accessible from outside
- **const methods**: Promise not to modify the object

6.3 Classes with Dynamic Memory

When a class manages dynamic memory, we need special care:

```cpp
#include <iostream>
using namespace std;

class IntArray {
private:
    int* data_;
    int size_;

public:
    // Constructor
    IntArray(int size) : size_(size) {
        data_ = new int[size_];
        for (int i = 0; i < size_; i++) {
            data_[i] = 0;
```

```cpp
        }
        cout << "Array of size " << size_ << " created\n";
    }

    // Destructor - MUST free memory!
    ~IntArray() {
        delete[] data_;
        cout << "Array destroyed\n";
    }

    // Access element
    int& operator[](int index) {
        return data_[index];
    }

    int getSize() const {
        return size_;
    }

    void print() const {
        for (int i = 0; i < size_; i++) {
            cout << data_[i] << " ";
        }
        cout << "\n";
    }
};

int main() {
    IntArray arr(5);
    arr[0] = 10;
    arr[1] = 20;
    arr[2] = 30;
    arr.print();

    return 0;
}  // Destructor automatically frees memory
```

Listing 13: A class with dynamic memory

To compile and run:

```
g++ -std=c++17 -Wall -o int_array int_array.cpp
./int_array
```

Expected output:

```
Array of size 5 created
10 20 30 0 0
Array destroyed
```

Key Concept

Why Destructors Matter:
Without a destructor:

```
IntArray arr(100);
// When arr goes out of scope, the object is destroyed
// BUT the allocated array is NOT freed - MEMORY LEAK!
```

With a proper destructor:

```
~IntArray() {
    delete[] data_;  // Frees the dynamic memory
}
// Now memory is properly cleaned up
```

6.4 The Rule of Three

Warning

The Rule of Three:
If your class manages a resource (like dynamic memory), and you need to define *any* of these three, you should define *all three*:

1. **Destructor** – to free the resource
2. **Copy Constructor** – to make a deep copy
3. **Copy Assignment Operator** – to assign with a deep copy

Why? Because the default versions do *shallow copy*, which causes problems with dynamic memory.

```
#include <iostream>
#include <algorithm>  // for std::copy
using namespace std;

class IntArray {
private:
    int* data_;
    int size_;

public:
    // 1. Constructor
    IntArray(int size) : size_(size) {
        data_ = new int[size_]();  // () means zero-initialize
    }

    // 2. Destructor
    ~IntArray() {
        delete[] data_;
```

```cpp
    }

    // 3. Copy Constructor (deep copy)
    IntArray(const IntArray& other) : size_(other.size_) {
        data_ = new int[size_];
        copy(other.data_, other.data_ + size_, data_);
        cout << "Copy constructor called\n";
    }

    // 4. Copy Assignment Operator (deep copy)
    IntArray& operator=(const IntArray& other) {
        if (this != &other) {  // Self-assignment check
            delete[] data_;    // Free old memory
            size_ = other.size_;
            data_ = new int[size_];
            copy(other.data_, other.data_ + size_, data_);
            cout << "Copy assignment called\n";
        }
        return *this;
    }

    int& operator[](int index) { return data_[index]; }
    int getSize() const { return size_; }

    void print() const {
        for (int i = 0; i < size_; i++)
            cout << data_[i] << " ";
        cout << "\n";
    }
};

int main() {
    IntArray a(5);
    a[0] = 10;
    a[1] = 20;
    a.print();              // 10 20 0 0 0

    IntArray b = a;         // Copy constructor
    b[0] = 99;
    a.print();              // 10 20 0 0 0 (unchanged)
    b.print();              // 99 20 0 0 0 (separate copy)

    IntArray c(3);
    c = a;                  // Copy assignment
    c.print();              // 10 20 0 0 0

    return 0;
}
```

Listing 14: The Rule of Three in action

To compile and run:

```
g++ -std=c++17 -Wall -o rule_of_three rule_of_three.cpp
./rule_of_three
```

Expected output:

```
Copy constructor called
10 20 0 0 0
99 20 0 0 0
Copy assignment called
10 20 0 0 0
```

Key Concept

Shallow vs Deep Copy:
Shallow Copy (default - BAD for dynamic memory):

```
a.data_ -----> [10][20][30]
                  ^
                  |
b.data_ ----------+
Both point to same array - DISASTER when destructor runs!
```

Deep Copy (what we want):

```
a.data_ -----> [10][20][30]
b.data_ -----> [10][20][30]  (separate copy)
Each has its own memory - SAFE!
```

Practice Exercise

Exercise 7: What happens if `IntArray` doesn't define a copy constructor, and we do this:

```
IntArray a(5);
a[0] = 42;
IntArray b = a;  // Uses default copy constructor
// When a and b are destroyed...
```

Hint: Think about what happens in the destructor!

7 Templates: Generic Programming

Story

Imagine you write a perfect `IntArray` class. Now your friend needs a `DoubleArray`. Do you copy-paste the entire class and change `int` to `double`?
That's terrible engineering! Every bug fix, every improvement, you'd have to apply twice (or three times, or ten times). There has to be a better way—and there is: **templates**.

7.1 Function Templates

```cpp
#include <iostream>
using namespace std;

// Without template - must write separate functions
int maxInt(int a, int b) {
    return (a > b) ? a : b;
}

double maxDouble(double a, double b) {
    return (a > b) ? a : b;
}

// With template - one function for all types!
template <typename T>
T maximum(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    cout << maximum(5, 10) << "\n";          // T = int
    cout << maximum(3.14, 2.71) << "\n";     // T = double
    cout << maximum('a', 'z') << "\n";       // T = char

    return 0;
}
```

Listing 15: Generic function with templates

To compile and run:

```
g++ -std=c++17 -Wall -o template_func template_func.cpp
./template_func
```

Expected output:

```
10
3.14
z
```

Key Concept

Template Syntax:

```
template <typename T>
returnType functionName(parameters) {
    // Use T as a type
}
```

- `typename T` declares a type parameter
- `T` can be used anywhere a type name would go
- The compiler generates a separate function for each type you use

7.2 Class Templates

```
#include <iostream>
#include <algorithm>
using namespace std;

template <typename T>
class DynamicArray {
private:
    T* data_;
    int size_;
    int capacity_;

    void resize() {
        capacity_ *= 2;
        T* newData = new T[capacity_];
        copy(data_, data_ + size_, newData);
        delete[] data_;
        data_ = newData;
    }

public:
    DynamicArray() : data_(new T[2]), size_(0), capacity_(2) {}

    ~DynamicArray() {
        delete[] data_;
    }

    void pushBack(const T& value) {
        if (size_ == capacity_) {
            resize();
        }
        data_[size_++] = value;
    }

    T& operator[](int index) {
```

```
        return data_[index];
    }

    int getSize() const { return size_; }

    void print() const {
        cout << "[";
        for (int i = 0; i < size_; i++) {
            if (i > 0) cout << ", ";
            cout << data_[i];
        }
        cout << "]\n";
    }
};

int main() {
    // Array of ints
    DynamicArray<int> numbers;
    numbers.pushBack(10);
    numbers.pushBack(20);
    numbers.pushBack(30);
    numbers.print();  // [10, 20, 30]

    // Array of strings
    DynamicArray<string> words;
    words.pushBack("Hello");
    words.pushBack("World");
    words.print();    // [Hello, World]

    return 0;
}
```

Listing 16: Generic dynamic array class

To compile and run:

```
g++ -std=c++17 -Wall -o dynamic_template dynamic_template.cpp
./dynamic_template
```

Expected output:

```
[10, 20, 30]
[Hello, World]
```

Key Concept

Using Class Templates:

```
DynamicArray<int> intArray;       // T = int
DynamicArray<double> doubleArray; // T = double
DynamicArray<string> stringArray; // T = string
```

The compiler generates a separate class for each type in the <> brackets.

Practice Exercise

Exercise 8: Write a template function `swap` that works for any type:

```
template <typename T>
void swap(T& a, T& b) {
    // Your code here
}

int main() {
    int x = 1, y = 2;
    swap(x, y);  // Should swap

    string s1 = "hello", s2 = "world";
    swap(s1, s2);  // Should also work
}
```

8 Building a Node Structure

Story

Everything we've learned today—pointers, dynamic memory, classes, templates—comes together in one simple yet powerful structure: the **Node**.
A node is the basic building block of *linked data structures*:

- Linked lists are chains of nodes
- Trees are branching structures of nodes
- Graphs are networks of nodes

Master the node, and you've laid the foundation for the rest of this course.

8.1 What is a Node?

Key Concept

A **node** is a structure that contains:

1. **Data**: The value being stored
2. **Pointer(s)**: Link(s) to other node(s)

```
template <typename T>
struct Node {
    T data;            // The value stored in this node
    Node<T>* next;     // Pointer to the next node

    // Constructor
    Node(const T& value) : data(value), next(nullptr) {}
```

```
};
```

Listing 17: Simple node structure

8.2 Creating and Linking Nodes

```
#include <iostream>
using namespace std;

template <typename T>
struct Node {
    T data;
    Node<T>* next;

    Node(const T& value) : data(value), next(nullptr) {}
};

int main() {
    // Create three nodes
    Node<int>* first = new Node<int>(10);
    Node<int>* second = new Node<int>(20);
    Node<int>* third = new Node<int>(30);

    // Link them together
    first->next = second;
    second->next = third;
    // third->next is already nullptr

    // Now we have: 10 -> 20 -> 30 -> nullptr

    // Traverse and print
    Node<int>* current = first;
    while (current != nullptr) {
        cout << current->data << " -> ";
        current = current->next;
    }
    cout << "nullptr\n";

    // IMPORTANT: Clean up memory!
    delete first;
    delete second;
    delete third;

    return 0;
}
```

Listing 18: Building a simple linked chain

To compile and run:

```
g++ -std=c++17 -Wall -o linked_nodes linked_nodes.cpp
./linked_nodes
```

Expected output:

```
10 -> 20 -> 30 -> nullptr
```

Key Concept

Visual Model of Linked Nodes:

```
first
  |
  v
+------+------+     +------+------+     +------+------+
| data | next-|---->| data | next-|---->| data | next-|----> nullptr
|  10  |      |     |  20  |      |     |  30  |      |
+------+------+     +------+------+     +------+------+

Each node is a separate memory allocation.
The 'next' pointer connects them into a chain.
```

```cpp
// Proper way to delete all nodes in a chain
Node<int>* current = first;
while (current != nullptr) {
    Node<int>* next = current->next;  // Save next before deleting
    delete current;
    current = next;
}
```

Listing 19: Better cleanup using loop

Warning

Common Node Mistakes:

1. **Forgetting to initialize `next` to `nullptr`:**

```cpp
Node* n = new Node(10);
// n->next is garbage! Always initialize to nullptr
```

2. **Losing access to nodes:**

```cpp
Node* head = new Node(1);
head = new Node(2);  // Lost the first node - memory leak!
```

3. **Not deleting all nodes:**

```cpp
Node* head = new Node(1);
head->next = new Node(2);
delete head;  // Leaked the second node!
```

Practice Exercise

Exercise 9: Write a function that counts the number of nodes in a chain:

```
template <typename T>
int countNodes(Node<T>* head) {
    // Your code here
}

int main() {
    Node<int>* head = new Node<int>(1);
    head->next = new Node<int>(2);
    head->next->next = new Node<int>(3);

    cout << countNodes(head);  // Should print 3
}
```

Exercise 10: Write a function that safely deletes all nodes in a chain:

```
template <typename T>
void deleteChain(Node<T>* head) {
    // Your code here
}
```

9 Summary and Key Takeaways

Key Concept

What You Learned Today:

1. **Compilation**: C++ source → object files → executable
2. **Memory**: Stack (automatic) vs Heap (manual with `new`/`delete`)
3. **Pointers**: Variables that store memory addresses
   - `&x` = address of x
   - `*p` = value at address p
   - Always initialize! Use `nullptr` when unsure
4. **References**: Aliases for variables (useful for function parameters)
5. **Parameter Passing**:
   - By value: copies (safe but slow for large objects)
   - By reference: direct access (fast, can modify original)
   - By pointer: address passing (allows null, can modify original)
6. **Dynamic Memory**:
   - `new` allocates, `delete` frees
   - Arrays need `delete[]`
   - Always free what you allocate!
7. **Classes**: Bundle data and functions
   - Constructor initializes
   - Destructor cleans up
   - Rule of Three for classes with resources
8. **Templates**: Write generic code that works for any type
9. **Nodes**: Basic building blocks for linked structures

9.1 Common Pitfalls Checklist

Mistake	Solution
Uninitialized pointer	Always initialize: `int* p = nullptr;`
Forgot to delete	Every `new` needs a `delete`
Wrong delete	`delete` for single, `delete[]` for arrays
Dangling pointer	Set to `nullptr` after `delete`
Shallow copy problem	Implement Rule of Three
Memory leak	Free all allocated memory before program ends

9.2 Practice Problems - Solutions

Exercise 1: Output is `15`. The pointer modifies the original variable.

Exercise 2: Bug: `p` is uninitialized. Fix: `int* p = new int;`

Exercise 3: Memory leak! The first `int(42)` is never freed. The second assignment overwrites the pointer before deleting.

Exercise 4: Output is `15 10`. Only `a` is passed by reference, so only it gets modified.

Exercise 5: Missing `delete[] data;` before return.

Exercise 6: Should be `delete[] p;` (array delete, not single-object delete).

Exercise 7: **Double free / corruption!** Both `a` and `b` will try to delete the same memory.

Exercise 8:

```cpp
template <typename T>
void swap(T& a, T& b) {
    T tmp = a;
    a = b;
    b = tmp;
}
```

Exercise 9:

```cpp
template <typename T>
int countNodes(Node<T>* head) {
    int count = 0;
    while (head != nullptr) {
        count++;
        head = head->next;
    }
    return count;
}
```

Exercise 10:

```cpp
template <typename T>
void deleteChain(Node<T>* head) {
    while (head != nullptr) {
        Node<T>* temp = head;
        head = head->next;
        delete temp;
    }
}
```

10 What's Next?

Story

You now have the tools. Next lecture, we'll use them to build our first real data structure: the **dynamic array** (like C++'s `std::vector`).
We'll implement:

- Resizable storage
- Efficient element access
- Automatic memory management

And we'll analyze its performance mathematically, introducing **amortized analysis**—a powerful technique for understanding algorithms that don't always take the same time.
Make sure you're comfortable with pointers, `new`/`delete`, and classes before next lecture. Review this material as needed. The rest of the course builds on these foundations!

"The only way to learn a new programming language is by writing programs in it."
— Dennis Ritchie (Creator of C)

Practice, practice, practice!